Text Files

Text files in C are straightforward and easy to understand. All text file functions and types in C come from the **stdio** library.

When you need text I/O in a C program, and you need only one source for input information and one sink for output information, you can rely on **stdin** (standard in) and **stdout** (standard out). You can then use input and output redirection at the command line to move different information streams through the program. There are six different I/O commands in <stdio.h> that you can use with stdin and stdout:

- **printf** - prints formatted output to stdout
- **scanf** - reads formatted input from stdin
- **puts** - prints a string to stdout
- **gets** - reads a string from stdin
- **putc** - prints a character to stdout
- **getc**, **getchar** - reads a character from stdin

The advantage of stdin and stdout is that they are easy to use. Likewise, the ability to redirect I/O is very powerful. For example, maybe you want to create a program that reads from stdin and counts the number of characters:

```
#include <stdio.h>
#include <string.h>

void main()
{
    char s[1000];
    int count=0;
     while (gets(s))
        count += strlen(s);
    printf("%d\n",count);
}
```

Enter this code and run it. It waits for input from stdin, so type a few lines. When you are done, press CTRL-D to signal end-of-file (eof). The gets function reads a line until it detects eof, then returns a 0 so that the while loop ends. When you press CTRL-D, you see a count of the number of characters in stdout (the screen). (Use **man gets** or your compiler's documentation to learn more about the gets function.)

Now, suppose you want to count the characters in a file. If you compiled the program to an executable named **xxx**, you can type the following:

```
xxx < filename
```

Instead of accepting input from the keyboard, the contents of the file named **filename** will be used instead. You can achieve the same result using **pipes**:

```
cat < filename | xxx
```

You can also redirect the output to a file:

```
xxx < filename > out
```

This command places the character count produced by the program in a text file named **out**.

Sometimes, you need to use a text file directly. For example, you might need

to open a specific file and read from or write to it. You might want to manage several streams of input or output or create a program like a text editor that can save and recall data or configuration files on command. In that case, use the text file functions in stdio:

- **fopen** - opens a text file
- **fclose** - closes a text file
- **feof** - detects end-of-file marker in a file
- **fprintf** - prints formatted output to a file
- **fscanf** - reads formatted input from a file
- **fputs** - prints a string to a file
- **fgets** - reads a string from a file
- **fputc** - prints a character to a file
- **fgetc** - reads a character from a file